



Neuron-Level Fuzzy Memoization in RNNs

Franyell Silfa

Universitat Politècnica de Catalunya
Barcelona, Spain
fsilfa@ac.upc.edu

Jose-Maria Arnau

Universitat Politècnica de Catalunya
Barcelona, Spain
jarnau@ac.upc.edu

Gem Dot

Universitat Politècnica de Catalunya
Barcelona, Spain
gdot@ac.upc.edu

Antonio González

Universitat Politècnica de Catalunya
Barcelona, Spain
antonio@ac.upc.edu

ABSTRACT

Recurrent Neural Networks (RNNs) are a key technology for applications such as automatic speech recognition or machine translation. Unlike conventional feed-forward DNNs, RNNs remember past information to improve the accuracy of future predictions and, therefore, they are very effective for sequence processing problems.

For each application run, each recurrent layer is executed many times for processing a potentially large sequence of inputs (words, images, audio frames, etc.). In this paper, we make the observation that the output of a neuron exhibits small changes in consecutive invocations. We exploit this property to build a neuron-level fuzzy memoization scheme, which dynamically caches the output of each neuron and reuses it whenever it is predicted that the current output will be similar to a previously computed result, avoiding in this way the output computations.

The main challenge in this scheme is determining whether the new neuron's output for the current input in the sequence will be similar to a recently computed result. To this end, we extend the recurrent layer with a much simpler Bitwise Neural Network (BNN), and show that the BNN and RNN outputs are highly correlated: if two BNN outputs are very similar, the corresponding outputs in the original RNN layer are likely to exhibit negligible changes. The BNN provides a low-cost and effective mechanism for deciding when fuzzy memoization can be applied with a small impact on accuracy.

We evaluate our memoization scheme on top of a state-of-the-art accelerator for RNNs, for a variety of different neural networks from multiple application domains. We show that our technique avoids more than 24.2% of computations, resulting in 18.5% energy savings and 1.35x speedup on average.

CCS CONCEPTS

• **Computer systems organization** → **Neural Networks**; • **Computing Methodologies** → **Machine Learning**;

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MICRO-52, October 12–16, 2019, Columbus, OH, USA

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-6938-1/19/10...\$15.00

<https://doi.org/10.1145/3352460.3358309>

KEYWORDS

Recurrent Neural Networks, Long Short Term Memory, Binary Networks, Memoization

ACM Reference format:

Franyell Silfa, Gem Dot, Jose-Maria Arnau, and Antonio González. 2019. Neuron-Level Fuzzy Memoization in RNNs. In *Proceedings of The 52nd Annual IEEE/ACM International Symposium on Microarchitecture, Columbus, OH, USA, October 12–16, 2019 (MICRO-52)*, 12 pages. <https://doi.org/10.1145/3352460.3358309>

1 INTRODUCTION

Recurrent Neuronal Networks (RNNs) represent the state-of-the-art solution for many sequence processing problems such as speech recognition [15], machine translation [34] or automatic caption generation [32]. Not surprisingly, data recently published in [20] show that around 30% of machine learning workloads in Google's datacenters are RNNs, whereas Convolutional Neuronal Networks (CNNs) only represent 5% of the applications. Unlike CNNs, RNNs use information of previously processed inputs to improve the accuracy of the output, and they can process variable length input/output sequences.

Although RNN training can be performed efficiently on GPUs [7], RNN inference is more challenging. The small batch size (just one input sequence per batch) and the data dependencies in recurrent layers severely constrain the amount of parallelism. Hardware acceleration is key for achieving high-performance and energy-efficient RNN inference and, to this end, several RNN accelerators have been recently proposed [17, 18, 22, 23].

Neurons in an RNN are recurrently executed for processing the elements in an input sequence. An analysis of the output results reveals that many neurons produce very similar outputs for consecutive elements in the input sequence. On average, the relative difference between the current and previous output of a neuron is smaller than 23% in our set of RNNs, whereas previous work in [28] has reported similar results. Since RNNs are inherently error tolerant [36], we propose to exploit the aforementioned property to save computations by using a neuron-level fuzzy memoization scheme. With this approach, the outputs of a neuron are dynamically cached in a local memoization buffer. When the next output is predicted to be extremely similar to the previously computed result, the neuron's output is read from the memoization buffer rather than recalculating it, avoiding all the corresponding computations and memory accesses.

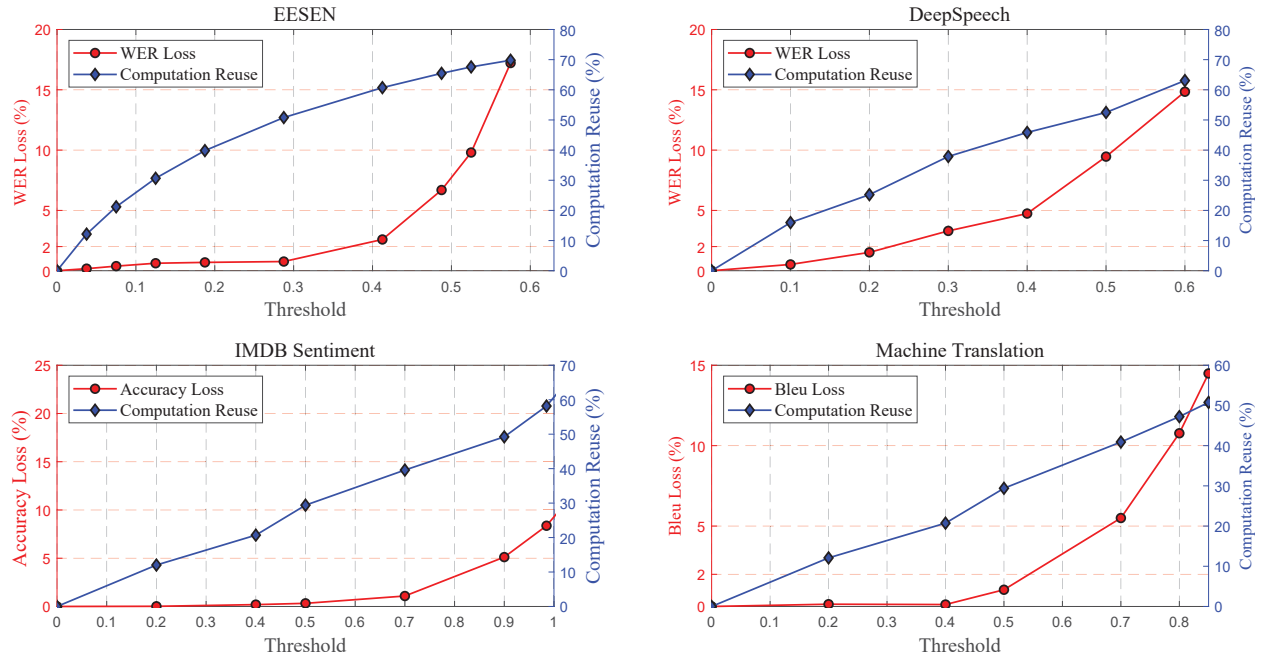


Figure 1: Accuracy loss of different RNNs versus the relative output error threshold using an oracle predictor. If the difference between the previous and current output predicted is smaller than the threshold, the memoized output is employed instead of calculating the new one.

Figure 1 shows the potential benefits of this memoization scheme by using an oracle that accurately predicts the relative difference between the next output of the neuron and the previous output stored in the memoization buffer. The memoized value is used when this difference is smaller than a given threshold, shown in the x-axis of Figure 1. As it can be seen, the RNNs can tolerate relative errors in the outputs of a neuron in the range of 30–50% with a negligible impact on accuracy. With these thresholds, a memoization scheme with an oracle predictor can save more than 30% of the computations.

A key challenge for our memoization scheme is how to predict the difference between the current output and the previous output stored in the memoization buffer, without performing all the corresponding neuron computations. To this end, we propose to extend each recurrent layer with a Bitwise Neural Network (BNN) [21]. We do this by reducing each input and weight to one bit that represents the sign as described in [11]. We found that BNN outputs are highly correlated with the outputs of the original recurrent layer, i.e. a similar BNN outputs indicates a high likelihood of having similar RNN output (although BNN outputs are very different to RNN outputs). The BNN is extremely small, hardware-friendly and very effective at predicting when memoization can be safely applied.

Note that by simply looking at the inputs, i.e. predicting that similar inputs will produce similar outputs, might not be accurate. Small changes in an input that is multiplied by a large weight will introduce a significant change in the output of the neuron. Our BNN approach takes into account both the inputs and the weights.

In short, we propose a neuron-level hardware-based fuzzy memoization scheme that works as follows. The output of a neuron in

the last execution is dynamically cached in a memoization table, together with the output of the corresponding BNN. For every new input in the sequence, the BNN is first computed and the result is compared with the BNN output stored in the memoization table. If the difference between the new BNN output and the cached output is smaller than a threshold, the neuron’s cached output is used as the current output, avoiding all the associated computations and memory accesses in the RNN. Otherwise, the neuron is evaluated and the memoization table is updated.

Note that only using the BNN would result in a large accuracy loss as reported elsewhere [27]. In this paper, we take a completely different approach and use the BNN to predict when memoization can be safely applied with negligible impact on accuracy. The inexpensive BNN is computed for every element of the sequence and every neuron, whereas the large RNN is evaluated on demand as indicated by the BNN. By doing so, we maintain high accuracy while saving more than 24.2% of RNN computations.

In this paper we make the following contributions:

- We provide an evaluation of the outputs of neurons in recurrent layers, and show that they exhibit small changes in consecutive executions.
- We propose a fuzzy memoization scheme that avoids more than 24.2% of neuron evaluations by reusing previously computed results stored in a memoization buffer.
- We propose the use of a BNN to determine when memoization can be applied with small impact on accuracy. We show that BNN and RNN outputs are highly correlated.

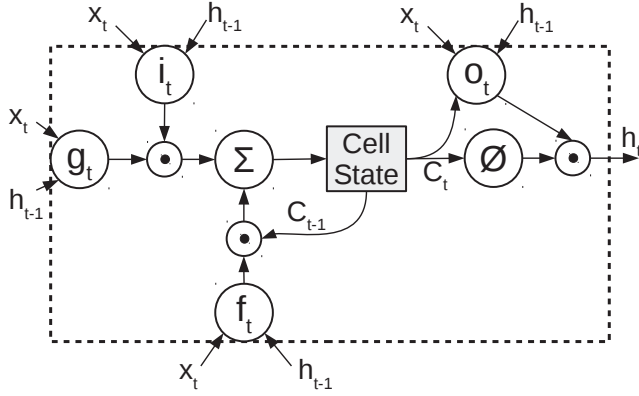


Figure 2: Structure of a LSTM cell. $\odot$ denotes an element-wise multiplication of two vectors. ϕ denotes the hyperbolic tangent.

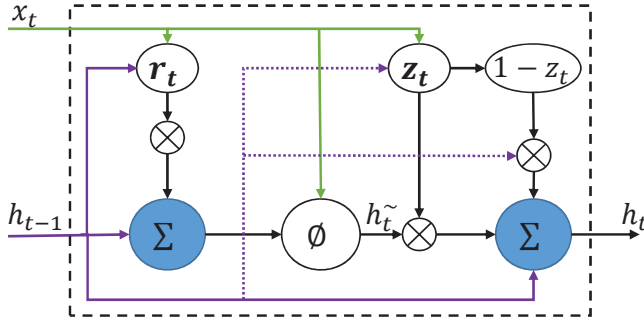


Figure 3: Structure of a GRU cell.

- We implement our neuron-level memoization scheme on top of a state-of-the-art RNN accelerator. The required hardware introduces a negligible area overhead, while it provides 1.35x speedup and 18.5% energy savings on average for several RNNs.

2 BACKGROUND

2.1 Recurrent Neural Networks

A Recurrent Neural Network (RNN) is a state-of-the-art machine learning approach that has achieved tremendous success in applications such as machine translation or video description. The key characteristic of RNNs is that they include loops, a.k.a. recurrent connections, that allow the information to persist from one time-step of execution to the next ones and, hence, they have the potential to use unbounded context information (i.e. past or future) to make predictions. Another important feature is that RNNs are recurrently executed for every element of the input sequence and, thus, they are able to handle input and output with variable length. Because of these characteristics, RNNs provide an effective framework for sequence-to-sequence applications (e.g. machine translation), where they outperform feed forward Deep Neural Networks (DNNs) [16, 29].

Basic RNN architectures can capture and exploit short term dependencies in the input sequence. However, capturing long term dependencies is challenging since useful information tend to dilute over time. In order to exploit long term dependencies, Long Short Term Memory (LSTM) [19] and Gated Recurrent Units (GRU) [10] networks were proposed. These types of RNNs represent the most successful and widely used RNN architectures. They have achieved tremendous success for a variety of applications such as speech recognition [5, 24], machine translation [9] and video description [32]. The next subsections provide further details on the structure and behavior of these networks.

2.1.1 Deep RNNs. RNNs are composed of multiple layers that are stacked together to create deep RNNs. Each of these layers consists of an LSTM or a GRU cell. In addition, these layers can be unidirectional or bidirectional. Unidirectional layers only use past information to make predictions, whereas bidirectional LSTM or GRU networks use both past and future context.

The input sequence (X) is composed of N elements, i.e. $X = [x_1, x_2, \dots, x_N]$, which are processed by an LSTM or GRU cell in the forward direction, i.e. from x_1 to x_N . For backward layers in bidirectional RNNs, the input sequence is evaluated in the backward direction, i.e. from x_N to x_1 .

2.1.2 LSTM Cell. Figure 2 shows the structure of an LSTM cell. The key component is the cell state (c_t), which is stored in the cell memory. The cell state is updated by using three fully connected single-layer neural networks, a.k.a. gates. The input gate (i_t , whose computations are shown in Equation 1) decides how much of the input information, x_t , will be added to the cell state. The forget gate (f_t , shown in Equation 2) determines how much information will be erased from the cell state (c_{t-1}). The updater gate (g_t , Equation 3) controls the amount of input information that is being considered a candidate to update the cell state (c_t). Once these three gates are executed, the cell state is updated according to Equation 4. Finally, the output gate (o_t , Equation 5) decides the amount of information that will be emitted from the cell to create the output (h_t).

Figure 4 shows the computations carried out by an LSTM cell. As it can be seen, a neuron in each gate has two types of connections: forward connections that operate on x_t and recurrent connections that take as input h_{t-1} . The evaluation of a neuron in one of these gates requires a dot product between weights in forward connections and x_t , and another dot product between weights in recurrent connections and h_{t-1} . Next, a peephole connection [13] and a bias are also applied, followed by the computation of an activation function, typically a sigmoid or hyperbolic tangent.

2.1.3 GRU Cell. Analogous to an LSTM cell, a GRU cell includes gates to control the flow of information inside the cell. However, GRU cells do not have an independent memory cell (i.e. cell state). As it can be seen in Figure 3, in a GRU cell the update gate (z_t) controls how much of the candidate information (g_t) is used to update the cell activation. On the other hand, the reset gate (r_t) modulates the amount of information that is removed from the previous computed state. Note that GRUs do not include an output gate and, hence, the whole state of the cell is exposed at each timestep. The computations carried out by each gate in a GRU cell are very similar to those in Equations 1, 2 and 3. We omit them for

$$i_t = \sigma(W_{ix}x_t + W_{ih}h_{t-1} + b_i) \quad (1)$$

$$f_t = \sigma(W_{fx}x_t + W_{fh}h_{t-1} + b_f) \quad (2)$$

$$g_t = \phi(W_{gx}x_t + W_{gh}h_{t-1} + b_g) \quad (3)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \quad (4)$$

$$o_t = \sigma(W_{ox}x_t + W_{oh}h_{t-1} + b_o) \quad (5)$$

$$h_t = o_t \odot \phi(c_t) \quad (6)$$

Figure 4: Computations of an LSTM cell. $\odot$, ϕ , and σ denote element-wise multiplication, hyperbolic tangent and sigmoid function respectively.

the sake of brevity, the exact details are provided in [10]. For the rest of the paper, we used the term RNN cell to refer to both LSTM and GRU cells.

2.2 Binarized Neural Networks

State-of-the-art DNNs typically consist of millions of parameters (a.k.a. weights) represented as floating point numbers using 32 or 16 bits and, hence, their storage requirements are quite large. Linear quantization may be used to reduce memory footprint and improve performance [20, 34]. In addition, real-time evaluation of DNNs requires a high energy cost. As an attempt to improve the energy-efficiency of DNNs, Binarized Neural Networks (BNNs) [11] or Bitwise Neural Networks [21] are a promising alternative to conventional DNNs. BNNs use one-bit weights and inputs that are constrained to +1 or -1. Typically, the binarization is done using the following function:

$$x^b = \begin{cases} +1 & \text{if } x \geq 0, \\ -1 & \text{otherwise,} \end{cases} \quad (7)$$

where x is either a weight or an input and x^b is the binarized value which is stored as 0 or 1. Regarding the output of a given neuron, its computation is analogous to conventional DNNs, but employing the binarized version of weights and inputs, as shown in Equation 8:

$$y_t^b = \sum w^b x_t^b \quad (8)$$

where w^b and x_t^b are the binarized weight and input vectors respectively. Note that evaluating the neuron output (y_t^b) only involves multiplications and additions that, with binarized operands, can be computed with XNORs and integer adders. BNN evaluation is orders of magnitude more efficient, in terms of both performance and energy, than conventional DNNs [11]. Nonetheless, DNNs and RNNs still deliver significantly higher accuracy than BNNs [27].

2.3 Fuzzy Memoization

Memoization is a well-known optimization technique used to improve performance and energy consumption that has been used both in software [2] and hardware [14]. In some applications, a given function is executed many times, but the inputs of different executions are not always different. Memoization exploits this fact

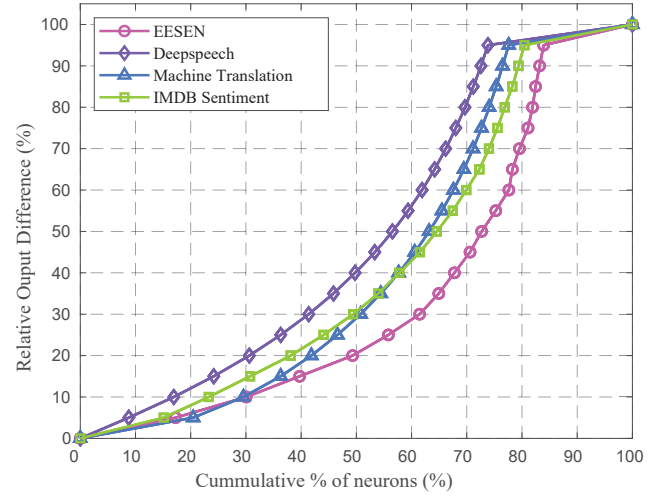


Figure 5: Relative change in neuron output between consecutive input elements.

to avoid these redundant computations by reusing the result of a previous evaluation. In general, the first time an input is evaluated, the result is cached in a memoization table. Subsequent evaluations probe the memoization table and reuse previously cached results if the current input matches a previous execution.

In a classical memoization scheme, a memoized value is only reused when it is known to be equal to the real output of the computation. However, for some applications such as multimedia [4], graphics [8], and neural networks [36], this scheme can be extended to tolerate a small loss in accuracy with negligible impact in the quality of the results, and is normally referred to as fuzzy memoization.

3 NEURON LEVEL MEMOIZATION

In this section, we propose a novel memoization scheme to reduce computations and memory accesses in RNNs. First, we discuss the main performance and energy bottlenecks on state-of-the-art hardware accelerators for RNN inference. Next, we introduce the key idea for our neuron-level fuzzy memoization technique. Finally, we describe the hardware implementation of our technique.

3.1 Motivation

As shown in Figure 4, RNN inference involves the evaluation of multiple single-layer feed-forward neural networks or gates that, from a computational point of view, consist of multiplying a weight matrix by an input vector (x_t for forward connections and h_{t-1} for recurrent connections). Typically, the number of elements in the weight matrices ranges from a few thousands to millions of elements and, thus, fetching them from on-chip buffers or main memory is one of the major sources of energy consumption. Not surprisingly, it accounts for up to 80% of the total energy consumption in state-of-the-art accelerators [30]. For this reason, a very effective way of saving energy in RNNs is to avoid fetching the synaptic weights. In addition, avoiding the corresponding computations also increases the energy savings. In this work, we leverage

$$\delta = \left| \frac{y_t^o - y_m}{y_t^o} \right| \quad (9)$$

$$y_t = \begin{cases} y_m & \text{if } \delta \leq \theta \\ y_t^o & \text{otherwise,} \end{cases} \quad (10)$$

$$y_m = \begin{cases} y_t^o & \text{if } \delta > \theta \\ \text{not updated} & \text{otherwise,} \end{cases} \quad (11)$$

Figure 6: Neuron Level memoization with Oracle Predictor. y_t is the neuron output. y_m corresponds to the memoized evaluation and y_t^o is the output of the Oracle predictor. δ , θ are the relative error and the maximum allowed output error respectively.

fuzzy memoization to selectively avoid neurons evaluations and, hence, to avoid their corresponding memory accesses and computations. For fuzzy memoization to be effective, applications must be tolerant to small errors and its hardware implementation must be simple. In the next sections, we show that RNNs are resilient to small errors in the outputs of the neurons, and we provide an efficient implementation of the memoization scheme that requires simple hardware support.

3.1.1 RNNs Redundancy. Memoization schemes rely on a high degree of redundancy in the computations. For RNNs, a key observation is that the output of a given neuron tends to change lightly between consecutive input elements. Note that RNNs are used in sequence processing problems such as speech recognition or video processing, where RNN inputs in consecutive time steps tend to be extremely similar. Prior work in [28] reports high similarity across consecutive frames of audio or video. Not surprisingly, our own numbers for our set of RNNs also support this claim. Figure 5 shows the relative difference between consecutive outputs of a neuron in our set of RNNs. As it can be seen, a neuron’s output exhibits small changes (less than 10%) for 25% of consecutive input elements. On average, consecutive outputs change by 23%. Furthermore, RNNs can tolerate small errors in the neuron output [36]. This observation is supported by data shown in Figure 1, where the accuracy curve shows the accuracy loss when the output of a neuron is reused using fuzzy memoization, for different thresholds (x-axis) that control the aggressiveness of the memoization scheme. For this study, the relative error (δ) between a predicted neuron output (y_t^p) and a previously cached neuron output (y_m) is used as the discriminating factor to decide whether the previous output is reused, as shown in Figure 6. To evaluate the potential benefits of a memoization scheme, the predicted value is provided by an Oracle predictor which is 100% accurate, i.e. its prediction is always equal to the neuron output ($y_t^p = y_t$). As shown in Figure 1, neurons can tolerate a relative output error between 0.3 and 0.5 without significantly affecting the overall network accuracy (i.e., accuracy loss smaller than 1%). On the other hand, the reuse curve shows the percentage of neuron computations that could be avoided through this memoization with an Oracle predictor. Note that by allowing neurons to have an output error between 0.3 to 0.5, at least 30% of the total network computations could be avoided.

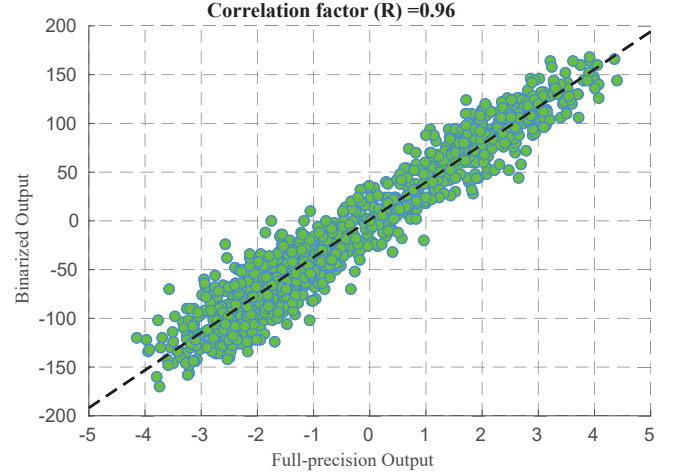


Figure 7: Outputs of the binarized neurons (y-axis) versus outputs of the full-precision neurons (x-axis) in EESN: an RNN for speech recognition. BNN and RNN outputs are highly correlated, showing a correlation coefficient of 0.96.

To achieve significant savings, the memoization scheme must add a small overhead to the system. The key challenge is how to approximate the behavior of the Oracle predictor with simple hardware, to decide when memoization can be safely applied with small impact on overall RNN accuracy. We describe an effective solution in the next section.

3.1.2 Binary Network Correlation. A key challenge for an effective fuzzy memoization scheme is to identify when the next neuron output will be similar to a previously computed (and cached) output. Note that having similar inputs does not necessarily result in similar outputs, as inputs with small changes might be multiplied by large weights. Our proposed approach is based on a Bitwise Neural Network (BNN). In particular, each fully-connected neural network (NN) is extended to an equivalent BNN, as described in Section 3.2. We use BNNs for two reasons. First, the outputs of a BNN and its corresponding original NN are highly correlated [6], i.e. a small change in a BNN output indicates that the neuron’s output in the original NN is likely to be similar. Second, BNNs can be implemented with extremely low hardware cost.

Regarding the correlation between BNN and RNN, Anderson et al. [6] show that the binarization approximately preserves the dot-products that a neural network performs for computations. Therefore, there should be a high correlation between the outputs of the full-precision neuron and the outputs of the corresponding binarized neuron. We have empirically validated the dot product preservation property for our set of RNNs. Figure 7 shows the linear correlation between RNN outputs and the corresponding BNN outputs for EESN network. Although the range of the outputs of the full-precision (RNN) and binarized (BNN) dot products are significantly different, their values exhibit a strong linear correlation (correlation coefficient of 0.96). On the other hand, Figure 8 shows the histogram of the correlation coefficients for the neurons

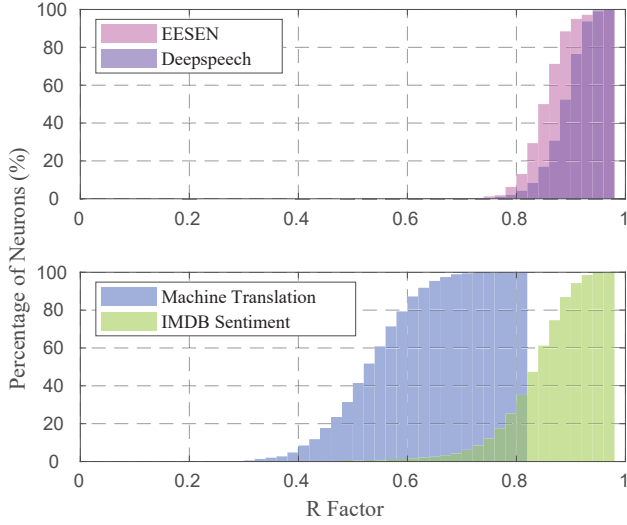


Figure 8: Correlation factor between the neuron output computed using full precision and the output computed with a BNN.

in four different RNNs. As it can be seen, correlation between binarized and full-precision neurons tend to be high for all the RNNs. More specifically, for the networks EESN, IMDB SENTIMENT, and DEEPSPEECH, 85% of the neurons have a linear correlation factor greater than 0.8 and for the Machine Translation network most of them have a correlation factor greater than 0.5. These results indicate that if the output of a binarized neuron shows very small changes with respect to a previously computed output, it is very likely that the full-precision neuron will also show small changes and, hence, memoization can be safely applied.

As shown in Equation 8, the output of a given neuron in a BNN can be computed with an N-bit XOR operation for bit multiplication and an integer adder to sum the resulting bits. These two operations are orders of magnitude cheaper than those required by the traditional data representation (i.e., FP16). Therefore, a BNN represents a low overhead and accurate manner to infer when the output of a neuron is likely to exhibit significant changes with respect to its recently computed outputs.

3.2 Overview

The target of our memoization scheme is to reuse a recently computed neuron output, y_m , as the output for the current timestep, y_t , provided that they are very similar. Reusing the cached neuron output avoids performing all the corresponding computations and memory accesses. To determine whether y_t will be similar to y_m , we use a BNN as a predictor.

In our memoization scheme, we extend the RNN with a much simpler BNN. The BNN model is created by mirroring the full precision trained model of an LSTM or GRU gate, as illustrated in Figure 9. More specifically, each neuron is binarized by applying the binarization function shown in Equation 7 to its corresponding set of weights. Therefore, in an gate, every neuron n with weights

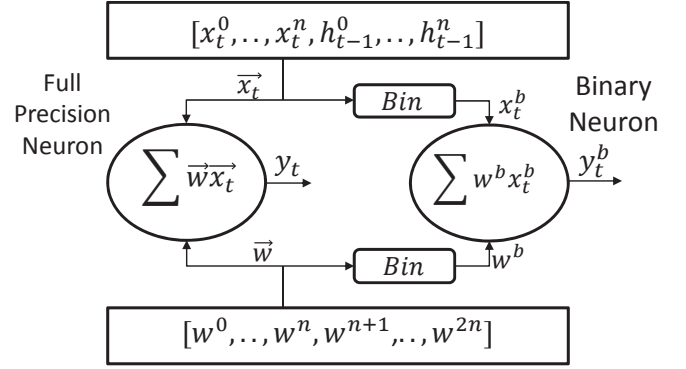


Figure 9: The figure illustrates how a binary neuron is created from a full precision neuron in the RNN network. *Bin* is the binarization function shown in Equation 7. Peephole, bias and activation functions are omitted for simplicity.

$$\epsilon_t^b = \left| \frac{y_t^b - y_m^b}{y_t^b} \right| \quad (12)$$

$$\delta_t^b = \sum_{i=m}^{i=t} \epsilon_i^b \quad (13)$$

$$y_t = \begin{cases} y_m & \text{if } \delta_t^b \leq \theta \\ \text{evaluate neuron} & \text{otherwise,} \end{cases} \quad (14)$$

$$y_m = \begin{cases} y_t & \text{if } \delta_t^b > \theta \\ \text{not updated} & \text{otherwise,} \end{cases} \quad (15)$$

$$y_m^b = \begin{cases} y_t^b & \text{if } \delta_t^b > \theta \\ \text{not updated} & \text{otherwise,} \end{cases} \quad (16)$$

$$\delta_t^b = \begin{cases} 0.0 & \text{if } \delta_t^b > \theta \\ \text{not updated} & \text{otherwise,} \end{cases} \quad (17)$$

Figure 10: Neuron level fuzzy memoization with binary network as predictor. y_t , y_m correspond to the neuron current and memoized output computed by the LSTM Network. y_t^b , y_m^b are the current and memoized output computed by the Binary Network. ϵ_t^b is the relative difference between BNN outputs. δ_t^b is the summation of relative differences in successive timesteps.

vector $\vec{w}$ is mirrored to a neuron n^b with weights vector $\vec{w}^b$ corresponding to the element-wise binarization of $\vec{w}$.

Our scheme stores recently computed outputs for the binary neuron n^b and its associated full precision neuron n . We refer to these memoized values as y_m^b and y_m respectively. On every timestep t , the binarized version of the neuron, n^b , is evaluated first obtaining y_t^b . Next, we compute the relative difference, ϵ_t^b , between y_t^b and y_m^b , i.e. the current and memoized outputs of the BNN, as shown in Equation 12. If ϵ_t^b is small, i.e. if the BNN outputs are similar, it means that the outputs of the full precision neuron are

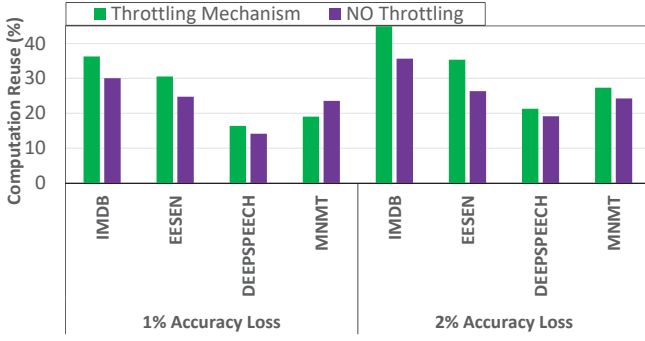


Figure 11: Computation reuse achieved by our BNN-based memoization scheme with and without the throttling mechanism, for accuracy losses of 1% and 2%. The throttling mechanism provides an extra 5% computation reuse on average for the same accuracy.

likely to be similar. As we discuss in Section 3.1.2, there is a high correlation between BNN and RNN outputs. In this case, we can reuse the memoized output y_m as the output of neuron n for the current timestep, avoiding all the corresponding computations. If the relative difference ϵ_t^b is significant, we compute the full precision neuron output, y_t , and update our memoization buffer, as shown in Equations 15, 16 and 17 so that these values can be reused in subsequent timesteps.

We have observed that applying memoization to the same neuron in a large number of successive timesteps may negatively impact accuracy, even though the relative difference ϵ_t^b in each individual timestep is small. We found that using a simple throttling mechanism can avoid this problem. More specifically, we accumulate the relative differences over successive timesteps where memoization is applied, as shown in Equation 13. We use the summation of relative differences, δ_t^b , to decide whether the memoized value is reused. As illustrated in Equation 14, the memoized value is only reused when δ_t^b is smaller or equal than a threshold θ . Otherwise, the full precision neuron is computed. This throttling mechanism avoids long sequences of timesteps where memoization is applied to the same neuron, since δ_t^b includes the differences accumulated in the entire sequence of reuses. Figure 11 shows that the throttling mechanism provides higher computation reuse for the same accuracy loss.

Figure 12 summarizes the overall memoization scheme, that is applied to the gates in an RNN cell as follows. For the first input element (x_0), i.e. the first timestep, the output values y_0^b (binarized version) and y_0 (in full precision) are computed for each neuron and stored in a memoization buffer. δ_0^b is set to zero. In the next timestep, with input x_1 , the value y_1^b is computed first by the BNN. Then, the relative error (ϵ_1^b) between y_1^b and the previously cached value, y_0^b , is computed and added to δ_0^b to obtain δ_1^b . Then, δ_1^b is compared with a threshold θ . If δ_1^b is smaller than θ , the cached value y_0 is reused, i.e. y_1 is assumed to be equal to y_0 , and δ_1^b is stored in the memoization buffer. On the contrary, if δ_1^b is larger than θ , the full precision neuron output y_1 is computed and its value is cached in a memoization buffer. In addition, y_1^b is also cached

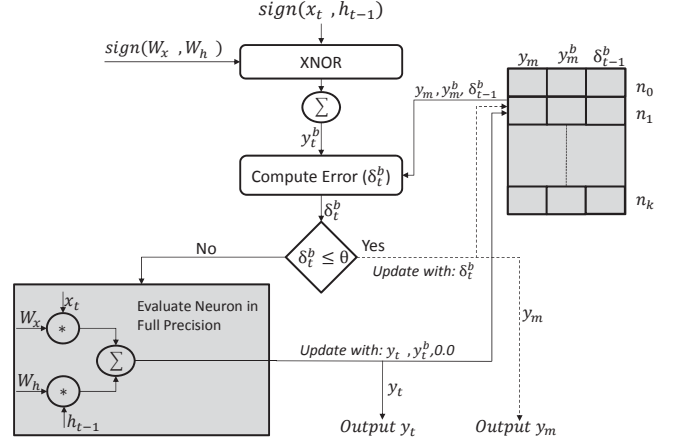


Figure 12: Fuzzy memoization scheme. W_x and W_h are the weights for the forward (x_t) and recurrent connections (h_{t-1}) respectively. y_t , y_m correspond to the current and cached neuron output computed in full precision. y_t^b , y_m^b are the current and cached output computed by the Binary Network. δ_t^b is the summation of relative differences in successive timesteps.

and δ_1^b is set to zero. This process is repeated for the remaining timesteps and for all the neurons in each gate.

3.2.1 Finding the threshold value. One of the key parameters in our scheme is the threshold θ . We perform an exploration of different values of θ for each RNN model by using the training set, obtaining accuracy and degree of computation reuse for each threshold value and RNN network. We then select the value that achieves highest computation reuse with the target accuracy loss (i.e. less than 1%) for each RNN model. This process is done just once for each model and once θ is determined, it can be used for inference on the test dataset.

3.3 Hardware Implementation

We implement the proposed memoization scheme on top of EPUR, a state-of-the-art RNN accelerator for low power mobile applications [30]. A high-level block diagram of this accelerator is shown in Figure 13. E-PUR is composed of four computational units that are tailored to the evaluation of each gate in an RNN cell, and a dedicated on-chip memory used to store intermediate results. In the next subsections, we outline the main components of the E-PUR architecture and detail the necessary hardware modifications required in order to support our fuzzy memoization scheme.

3.3.1 Hardware Baseline. In E-PUR each of the Computation Units (CUs), shown in Figure 14, are composed of a dot product unit (DPU), a Multi-functional Unit (MU) and buffers to store the weights and inputs. The DPU is used to evaluate the matrix vector multiplications between the weights and inputs (i.e. x_t and h_{t-1}) whereas the MU is used to compute activation functions and scalar operations. Note that in E-PUR computations can be performed using 32 or 16 bits floating points operations.

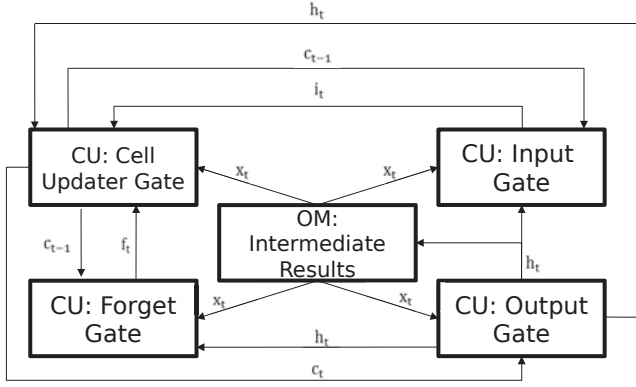


Figure 13: Overview of E-PUR architecture which consist of 4 Computation Units (CU) and an on-chip memory (OM).

In E-PUR, while evaluating an RNN cell, all the gates are computed in parallel for each input element. On the contrary, the neurons in each gate are evaluated in a sequential manner for the forward and recurrent connections. The following steps are executed in order to compute the output value (y_t) for a given neuron (i.e. n_i). First, the input and weight vectors formed by the recurrent and forward connections (i.e. x_t and h_{t-1}) are split into K sub-vectors of size N . Then, two sub-vectors of size N are loaded from the input and weight buffer respectively and the dot product between them is computed by the DPU, which also accumulates the result. Next, the steps are repeated for the next k^{th} sub-vector and its result is added to the previously accumulated dot products. This process is repeated until all K sub-vectors are evaluated and added together. Once the output value y_t is computed, the DPU sends it to the MU where bias and peephole calculations are performed. Finally, the MU computes the activation function and stores the result in the on-chip memory for intermediate results. Note that once the DPU sends a value to the MU, it will continue with the evaluation of the next neuron output, hence, overlapping the computations executed by the MU and DPU since they are independent. Finally, these steps are repeated until all the neurons in the gate (for all cells) are evaluated for the current input element.

3.3.2 Support for Fuzzy Memoization . In order to perform fuzzy memoization through a BNN, two modifications are done to each CU in E-PUR. First, the weight buffer is split into two buffers: one buffer is used to store the weight signs (sign buffer) and the other is used to store the remaining bits of the weights. Note that the sign buffer is always accessed to compute the output of the binary network (y_t^b) whereas the remaining bits are only accessed if the memoized value (y_m) is not reused. The binarized weights are stored in a small memory which has low energy cost but, as a consequence of splitting the weight buffer, its area increases a bit (less than one percent).

The second modification to the CUs is the addition of the fuzzy memoization unit (FMU) which is used to evaluate the binary network and to perform fuzzy memoization. This unit takes as input two size- T vectors (i.e., number of neurons in an RNN cell). The first vector is a weight vector loaded from the sign buffer whereas

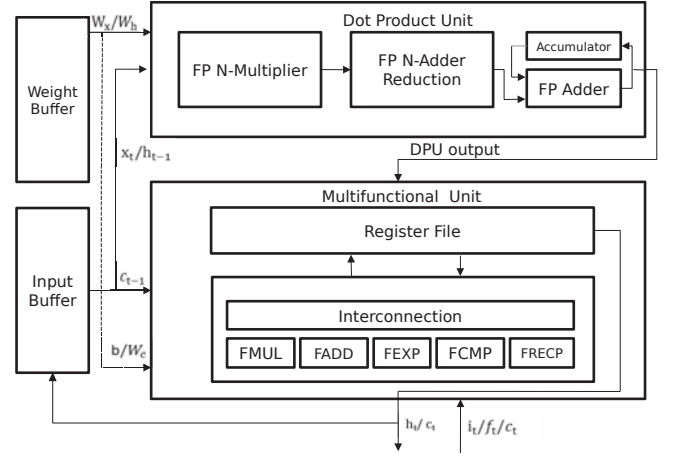


Figure 14: Structure of E-PUR Computation Unit.

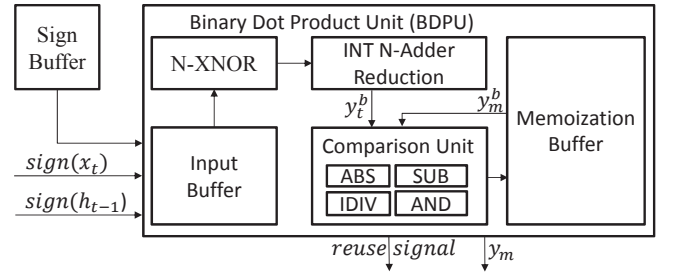


Figure 15: Structure of the Fuzzy Memoization Unit (FMU).

the other is created as the concatenation of the forward (x_t) and the recurrent connections (h_{t-1}).

As shown in Figure 15, the main components of the FMU are the BDPU that computes the binary dot product and the comparison unit (CMP) which decides when to reuse a memoized value. In addition, the FMU includes a buffer (memoization buffer) which stores the δ_t^b for every neuron and the latest evaluation of the neurons by the full precision and binary networks. BNN neurons (i.e. binary dot product) are evaluated using a bitwise XNOR operation and an adder reduction tree to gather the resulting bit vector. In the CMP unit, the relative error (δ_t^b) is computed using integer and fixed-point arithmetic.

The steps to evaluate the RNN cell, described in Section 3.3.1, are executed in a slightly different manner to include the fuzzy memoization scheme. First, the binarized input and weight vectors for a given neuron in a gate are loaded into an FMU from the input and sign buffers respectively. Next, the BDPU computes the dot product and sends the result (y_t^b) to the comparison unit (CMP). Then, the CMP loads the previously cached values y_m^b and δ_{t-1}^b from the memoization buffer and it uses them to compute the relative error (ϵ_t^b) and the δ_t^b . Once δ_t^b is computed, it is compared with a threshold (θ) to determine whether the full precision neuron needs to be evaluated or the previously cached value is reused instead. In the case that δ_t^b is greater than θ , an evaluation in full precision is triggered. In that regard, the DPU is signaled to start the full

Table 1: RNN Networks used for the experiments.

Network	App Domain	Cell Type	Layers	Neurons	Base Accuracy	Reuse	Dataset
IMDB Sentiment [12]	Sentiment Classification	LSTM	1	128	86.5%	36.2%	IMDB dataset
DeepSpeech2 [5]	Speech Recognition	GRU	5	800	10.24 WER	16.4%	LibriSpeech
EESN [24]	Speech Recognition	BiLSTM	10	320	23.8 WER	30.5%	Teddlum V1
MNMT [9]	Machine Translation	LSTM	8	1024	29.8 Bleu	19.0%	WMT'15 En → Ge

Table 2: Configuration Parameters.

E-PUR	
Parameter	Value
Technology	28 nm
Frequency	500 MHz
Intermediate Memory	6 MiB
Weight Buffer	2 MiB per CU
Input Buffer	8 KiB per CU
DPU Width	16 operations
Memoization Unit	
BDPU Width	2048 bits
Latency	5 cycles
Integer Width	2 bytes
Memoization Buffer	8 KiB

precision evaluation which is done following the steps described in Section 3.3.1. After the full precision evaluation, the values y_t , y_t^b , and 0.0 are cached in the memoization table corresponding to y_m , y_m^b , and δ_t^b respectively. On the other hand, if memoization can be applied (i.e. δ_t^b is smaller than the maximum allowed error), δ_t^b is updated in the memoization table and the memoized value (y_m) is sent directly to the MU (bypassing the DPU), so the full precision evaluation of the neuron is avoided. Finally, these steps are repeated until all the neurons in a gate are evaluated for the current input element. Since LSTM or GRU gates are processed by independent CUs, the above process is executed concurrently by all gates.

4 EVALUATION METHODOLOGY

We use a cycle-level simulator of E-PUR customized to model our scheme as described in Section 3.3.2. This simulator estimates the total energy consumption (static and dynamic) and execution time of the LSTM networks. The different pipeline components were implemented in Verilog and we synthesized them using the Synopsys Design Compiler to obtain their delay and energy consumption. Furthermore, we used a typical process corner with voltage of 0.78V. We employed CACTI [26] to estimate the delay and energy consumption (static and dynamic) of on-chip memories. Finally, to estimate timing and energy consumption of main memory we used MICRON's memory model [25]. We model 4 GB of LPDDR4 DRAM.

In order to set the clock frequency, the delays reported by Synopsys Design Compiler and CACTI are used. We set a clock frequency that allows most hardware structures to operate at one clock cycle. In addition, we evaluated alternative frequency values in order to minimize energy consumption.

Regarding the memoization unit, the configuration parameters are shown in Table 2. Since E-PUR supports large LSTM networks, the memoization unit is designed to match the largest models supported by E-PUR. This unit has a latency of 5 clock cycles for the largest supported LSTM networks. In this unit, integer and fixed point operations are used to perform most computations. The memoization buffer is modeled as 8KiB scratch-pad eDRAM.

The remaining configuration parameters of the accelerator used in our experiments are shown in Table 2. We strive to select an energy-efficient configuration for all the neural networks in Table 1. Because the baseline accelerator is designed to accommodate large LSTM networks, some of its on-chip storage and functional units might be oversized for some of our RNNs. In this case, unused on-chip memories and functional units are power gated when not needed.

As for benchmarks, we use four modern LSTM networks which are described in Table 1. Our selection includes RNNs for popular application such as speech recognition, machine translation and image description. These networks have different number of internal layers and neurons. We include both bidirectional (EESN) and unidirectional networks (the other three). On the other hand, the length of the input sequence is also different for each RNN and it ranges from 20 to a few thousand input elements.

The software implementation of the networks was done in Tensorflow [1]. We used the network models and the test set provided in [9, 12, 24, 33] for each RNN. The original accuracy for each RNN is listed in Table 1, and the accuracy loss is later reported as the absolute loss with respect to this baseline accuracy.

5 EXPERIMENTAL RESULTS

This section presents the evaluation of the proposed fuzzy memoization technique for RNNs, implemented on top of E-PUR [30]. We refer to it as E-PUR+BM. First, we present the percentage of computation reuse and the accuracy achieved. Second, we show the performance and energy improvements, followed by an analysis of the area overheads of our technique.

Figure 16 shows the percentage of computation reuse achieved by the BNN and the Oracle predictors. The percentage of computation reuse indicates the percentage of neuron evaluations avoided due to fuzzy memoization. For accuracy losses smaller than 2%, the BNN obtains a percentage of computation reuse extremely similar to the Oracle. The networks EESN and IMDB are highly tolerant to errors in neuron's outputs, thus, for these networks, our memoization scheme achieves reuse percentages of up to 40% while having an accuracy loss smaller than 3%. Note that, for classification problems, BNNs achieve an accuracy close to the state-of-the-art [27] and, hence, it is not surprising that the BNN predictor is highly accurate for approximating the neuron output. For DeepSpeech (speech

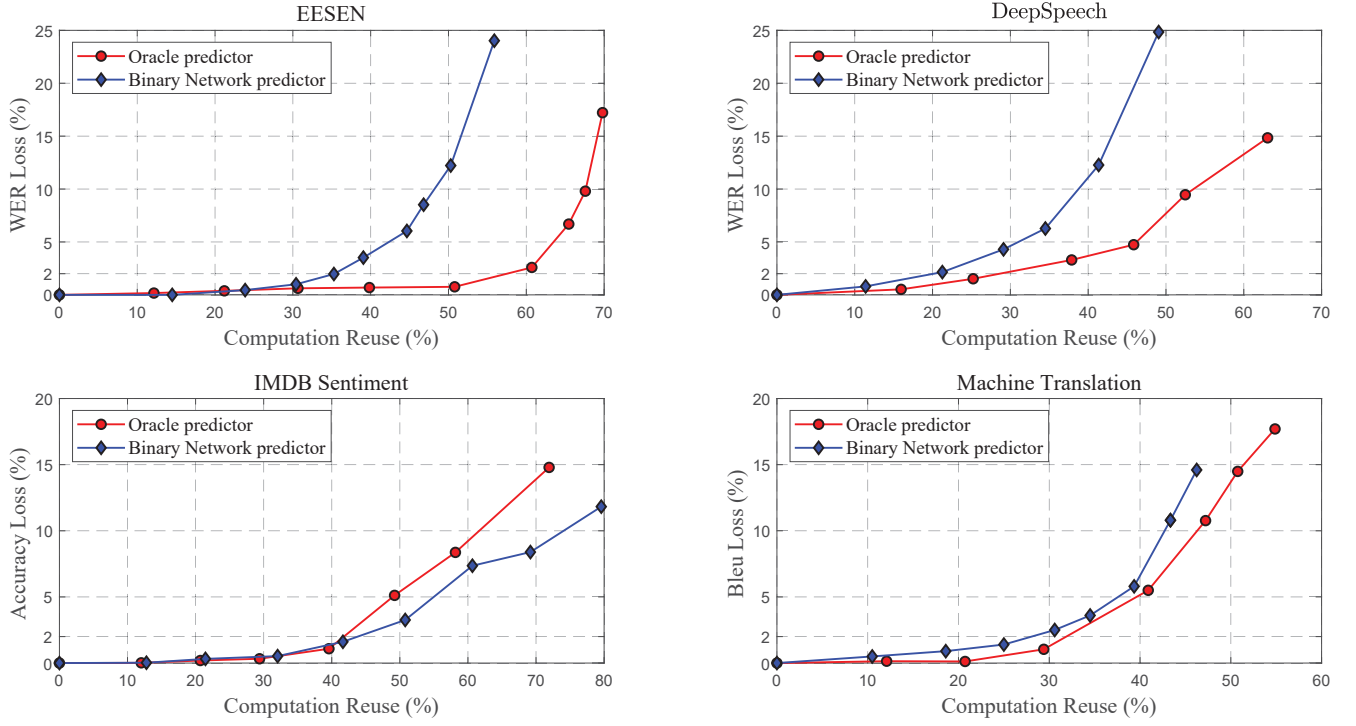


Figure 16: Percentage of computations that could be reused versus accuracy loss using Fuzzy Neuron Level Memoization with an Oracle and a Binary Network as predictors for several LSTM networks.

recognition) the reuse percentage is up to 20% for accuracy losses smaller than 2%. In this network, the input sequence tends to be large (i.e, 900 elements on average). As the reuse is increased, the error introduced to the output sequence of a neuron persists for a larger number of elements. Therefore, the introduced error will have a bigger impact both in the evaluation of the current layer, due to the recurrent connections, and the following layers. As a result, the overall accuracy of the network decreases faster. For MNMT (machine translation) the BNN predictor and the oracle achieve similar reuse versus accuracy trade-off for up to 23% of computation reuse. Note that, for this network, the linear correlation between the BNN and the full precision neuron output is typically lower than for the other networks in the benchmark set.

Figure 17 shows the energy savings and computation reuse achieved by our scheme, for different thresholds of accuracy loss. For a conservative loss of 2%, the average energy saving is 25.5%, whereas the reuse percentage is 31%. In this case, the networks DeepSpeech and MNMT and have similar energy savings, whereas the networks IMDB and EESN exhibit the largest savings since they are more tolerant to errors in the neuron output. For an extremely conservative 1% of accuracy loss, the computation reuse and energy saving are 24.2% and 18.5% on average respectively. EESN and DeepSpeech achieve 25.32% and 12.23% energy savings respectively for a 1% accuracy loss. Regarding the machine translation network (MNMT), the energy savings for 1% and 2% accuracy loss are 15.17% and 23.46% respectively.

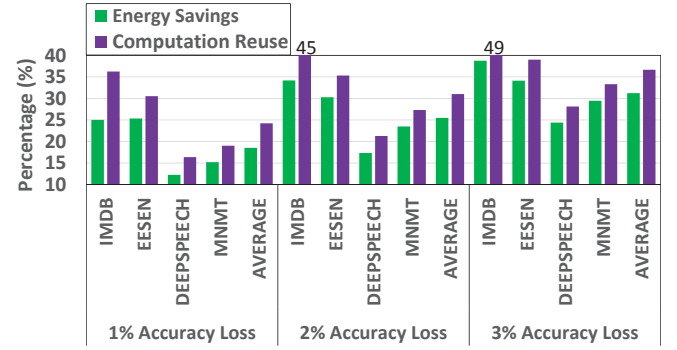


Figure 17: Energy savings and computation reuse of E-PUR+BM with respect to the baseline.

Regarding the sources of energy savings, Figure 18 reports the energy breakdown, including static and dynamic energy, for the baseline accelerator and E-PUR+BM, for an accuracy loss of 1%. The sources of energy consumption are grouped into on-chip memories ("scratch-pad" memories), pipeline components ("operations", i.e. multipliers), main memory (LPDDR4) and the energy consumed by our FMU component. Note that most of the energy consumption is due to the scratch-pad memories and the pipeline components and, as it can be seen, both are reduced when using our memoization scheme. In E-PUR+BM, each time a value from the memoization buffer is reused, we avoid accessing all the neuron's weights and

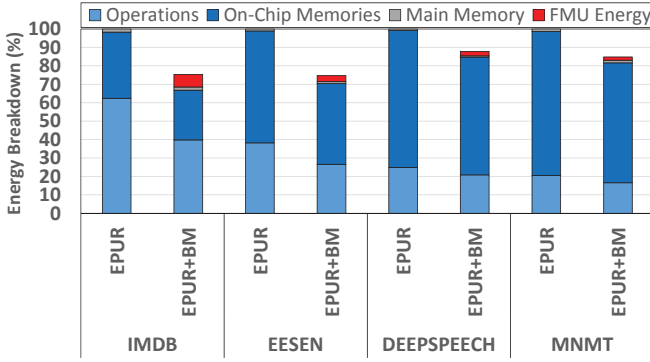


Figure 18: Energy breakdown for E-PUR and EPUR+BM. FMU Energy is the overhead due to the memoization scheme.

the input buffers, achieving significant energy savings. In addition, since the extra buffers used by E-PUR+BM are fairly small (i.e. 8 KB), the energy overhead due to the memoization scheme is negligible. The energy consumption due to the operations is also reduced, as the memoization scheme avoids neuron’s computations. Furthermore, the leakage of scratch-pad and operations are also reduced due to the speedups achieved by the memoization scheme. Finally, the energy consumption due to accessing main memory is not affected by our technique since both, E-PUR and E-PUR+BM, must access main memory to load all the weights once for each input sequence.

Figure 19 shows the performance improvements for the different RNNs. On average, an speedup of 1.35x is obtained for a 1% accuracy loss, whereas accuracy losses of 2% and 3% achieve improvements of 1.5x and 1.67x respectively. The performance improvement comes from avoiding the dot product computations for the memoized neurons. Therefore, the larger the degree of computation reuse the bigger the performance improvement. Note that the memoization scheme introduces an overhead of 5 cycles per neuron (see Table 2), mainly due to the evaluation of the binarized neuron. In case the full precision neuron evaluation can be avoided, our scheme saves between 16 and 80 cycles depending on the RNN. Therefore, configurations with low degree of computation reuse, like Deepspeech at 1% accuracy loss, exhibit smaller speedups due to the overhead of the memoization scheme. On the other hand, RNNs that exhibit higher computation reuse, such as ESEN at 2% accuracy loss, achieve an speedup of 1.55x.

E-PUR has an area of 64.6 mm^2 , whereas E-PUR+BM requires 66.8 mm^2 (4% area overhead). The largest overhead contribution (3%) is in the extra scratch-pad memory required by the memoization unit.

6 RELATED WORK

Increasing energy-efficiency and performance of LSTM networks has attracted the attention of the architectural community in recent years [17, 18, 22, 23]. Most of these works employ pruning and compression techniques to improve performance and reduce energy consumption. Furthermore, linear quantization is employed to decrease the memory footprint. On the contrary, our technique

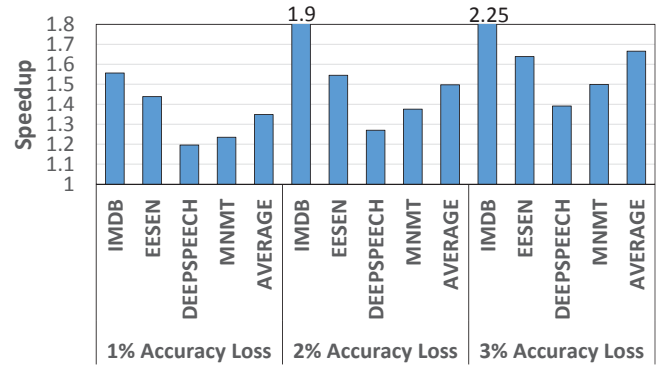


Figure 19: Speedup of E-PUR+BM over the baseline (E-PUR).

improves energy-efficiency by relying solely on computation reuse at the neuron level. To the best of our knowledge, this is the first work using a BNN as a predictor for a fuzzy memoization scheme. BNNs have been used previously [11, 21, 27] as standalone networks, whereas we employs BNNs in conjunction with the LSTM network to evaluate neurons on demand.

Fuzzy memoization has been extensively researched in the past and has been implemented both in hardware and software. Hardware schemes to reuse instructions have been proposed in [3, 8, 14, 31]. Alvarez et al. [4] presented a fuzzy memoization scheme to improve performance of floating point operations in multimedia applications. In their scheme floating point operations are memoized using a hash of the source operands, whereas in our technique, a whole function (neuron inference) is memoized based on the values predicted by a BNN.

Finally, software schemes to memoize entire functions have been presented in the past [2, 35]. These schemes are tailored to general purpose programs whereas our scheme is solely focused in LSTM networks, since it exploits the intrinsic error tolerance of LSTM networks.

7 CONCLUSIONS

In this paper, we have shown that 25% of neurons in an LSTM network change their output value by less than 10%. This motivated us to propose a fuzzy memoization scheme to save energy and time. A major challenge to perform neuron level fuzzy memoization is to predict, in a simple and accurate manner, whether the output of a given neuron will be similar to a previously computed and cached value. To this end, we propose to use a Binarized Neural Network (BNN) as a predictor, based on the observation that the fully precision output of a neuron is highly correlated with the output of the corresponding BNN. We show that a BNN predictor achieves 24.2% computation reuse on average, which is very similar to the results obtained with an Oracle predictor. We have implemented our technique on top of E-PUR, a state-of-the-art accelerator for LSTM networks. Results show that our memoization scheme achieves significant time and energy savings with minimal impact in the accuracy of the RNNs. When compared with the E-PUR accelerator, our scheme achieves 18.5% energy savings on average, while providing 1.35x speedup at the expense of a minor accuracy loss.

ACKNOWLEDGMENTS

This work has been supported by the the CoCoUnit ERC Advanced Grant of the EU's Horizon 2020 program (grant No 833057), the Spanish State Research Agency under grant TIN2016-75344-R (AEI/FEDER, EU), the ICREA Academia program, and the Fundaci3n Carolina and PUCMM by a scholarship.

REFERENCES

- [1] Martin Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Gregory S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian J. Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal J3zefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dan Man3, Rajat Monga, Sherry Moore, Derek Gordon Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul A. Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda B. Vi3gas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems. *CoRR abs/1603.04467* (2016). <http://arxiv.org/abs/1603.04467>
- [2] Umut A. Acar, Guy E. Blelloch, and Robert Harper. 2003. Selective Memoization. *SIGPLAN Not.* 38, 1 (Jan. 2003), 14–25. <https://doi.org/10.1145/640128.604133>
- [3] Carlos 3lvarez, Jes3s Corbal, Esther Salami, and Mateo Valero. 2001. On the Potential of Tolerant Region Reuse for Multimedia Applications (*ICS '01*). 218–228. <https://doi.org/10.1145/377792.377835>
- [4] Carlos 3lvarez, Jesus Corbal, and Mateo Valero. 2005. Fuzzy Memoization for Floating-Point Multimedia Applications. *IEEE Trans. Comput.* 54, 7 (July 2005), 922–927. <https://doi.org/10.1109/TC.2005.119>
- [5] Dario Amodei, Rishita Anubhai, Eric Battenberg, Carl Case, Jared Casper, Bryan Catanzaro, Jingdong Chen, Mike Chrzanowski, Adam Coates, Greg Diamos, Erich Elsen, Jesse Engel, Linxi Fan, Christopher Fougner, Tony Han, Awni Y. Hannun, Billy Jun, Patrick LeGresley, Libby Lin, Sharan Narang, Andrew Y. Ng, Sherjil Ozair, Ryan Prenger, Jonathan Raiman, Sanjeev Satheesh, David Seetapun, Shubho Sengupta, Yi Wang, Zhiqian Wang, Chong Wang, Bo Xiao, Dani Yogatama, Jun Zhan, and Zhenyao Zhu. 2015. Deep Speech 2: End-to-End Speech Recognition in English and Mandarin. *CoRR abs/1512.02595* (2015). <http://arxiv.org/abs/1512.02595>
- [6] Alexander G. Anderson and Cory P. Berg. 2017. The High-Dimensional Geometry of Binary Neural Networks. *CoRR abs/1705.07199* (2017). [arXiv:1705.07199](http://arxiv.org/abs/1705.07199)
- [7] Jeremy Appleyard, Tomas Kocisky, and Phil Blunsom. 2016. Optimizing Performance of Recurrent Neural Networks on GPUs. *arXiv preprint arXiv:1604.01946* (2016).
- [8] Jose-Maria Arnau, Joan-Manuel Parcerisa, and Polychronis Xekalakis. 2014. Eliminating redundant fragment shader executions on a mobile GPU via hardware memoization. *2014 ACM/IEEE 41st International Symposium on Computer Architecture (ISCA)* (2014), 529–540.
- [9] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. 2017. Massive Exploration of Neural Machine Translation Architectures. *CoRR abs/1703.03906* (2017). <http://arxiv.org/abs/1703.03906>
- [10] Kyunghyun Cho, Bart van Merri3nboer, 3aęlar G3l3chre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *CoRR abs/1406.1078* (2014). [arXiv:1406.1078](http://arxiv.org/abs/1406.1078)
- [11] Matthieu Courbariaux, Itay Hubara, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2016. Binarized neural networks: Training deep neural networks with weights and activations constrained to+ 1 or-1. *arXiv preprint arXiv:1602.02830* (2016).
- [12] Andrew M. Dai and Quoc V. Le. 2015. Semi-supervised Sequence Learning. *CoRR abs/1511.01432* (2015). <http://arxiv.org/abs/1511.01432>
- [13] Felix A Gers and J3rgen Schmidhuber. 2000. Recurrent nets that time and count. In *Neural Networks, 2000. IJCNN 2000, Proceedings of the IEEE-INNS-ENNS International Joint Conference on*, Vol. 3. IEEE, 189–194.
- [14] Antonio Gonz3lez, Jordi Tubella, and Carlos Molina-Clemente. 1999. Trace-Level Reuse. In *ICPP*. 30–.
- [15] A. Graves, A. r. Mohamed, and G. Hinton. 2013. Speech recognition with deep recurrent neural networks. In *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*. 6645–6649. <https://doi.org/10.1109/ICASSP.2013.6638947>
- [16] Klaus Greff, Rupesh K Srivastava, Jan Koutnik, Bas R Steunebrink, and J3rgen Schmidhuber. 2016. LSTM: A search space odyssey. *IEEE transactions on neural networks and learning systems* (2016).
- [17] Yijin Guan, Zhihang Yuan, Guangyu Sun, and Jason Cong. 2017. FPGA-based accelerator for long short-term memory recurrent neural networks. In *Design Automation Conference (ASP-DAC), 2017 22nd Asia and South Pacific*. IEEE, 629–634.
- [18] Song Han, Junlong Kang, Huizi Mao, Yiming Hu, Xin Li, Yubin Li, Dongliang Xie, Hong Luo, Song Yao, Yu Wang, Huazhong Yang, and William (Bill) J. Dally. 2017. ESE: Efficient Speech Recognition Engine with Sparse LSTM on FPGA. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA '17)*. ACM, New York, NY, USA, 75–84. <https://doi.org/10.1145/3020078.3021745>
- [19] Sepp Hochreiter and J3rgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.
- [20] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Daniel Killebrew, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthiu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-Datacenter Performance Analysis of a Tensor Processing Unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA '17)*. ACM, New York, NY, USA, 1–12. <https://doi.org/10.1145/3079856.3080246>
- [21] Minje Kim and Paris Smaragd3s. 2016. Bitwise neural networks. *arXiv preprint arXiv:1601.06071* (2016).
- [22] Minjae Lee, Kyuyeon Hwang, Jinhwan Park, Sungwook Choi, Sungho Shin, and Wonyong Sung. 2016. FPGA-based low-power speech recognition with recurrent neural networks. In *Signal Processing Systems (SiPS), 2016 IEEE International Workshop on*. IEEE, 230–235.
- [23] Sicheng Li, Chunpeng Wu, Hai Li, Boxun Li, Yu Wang, and Qinru Qiu. 2015. Fpga acceleration of recurrent neural network based language model. In *Field-Programmable Custom Computing Machines (FCCM), 2015 IEEE 23rd Annual International Symposium on*. IEEE, 111–118.
- [24] Yajie Miao, Mohammad Gowayyed, and Florian Metze. 2015. EESN: End-to-end speech recognition using deep RNN models and WFST-based decoding. In *Automatic Speech Recognition and Understanding (ASRU), 2015 IEEE Workshop on*. IEEE, 167–174.
- [25] Micron Inc. [n. d.]. TN-53-01: LPDDR4 System Power Calculator. <https://www.micron.com/support/tools-and-utilities/power-calc>
- [26] Naveen Muralimanohar, Rajeev Balasubramanian, and Norman P Jouppi. 2009. CACTI 6.0: A tool to model large caches. *HP Laboratories* (2009), 22–31.
- [27] Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. 2016. Xnor-net: Imagenet classification using binary convolutional neural networks. In *European Conference on Computer Vision*. Springer, 525–542.
- [28] Marc Riera, Jose-Maria Arnau, and Antonio Gonz3lez. 2018. Computation reuse in DNNs by exploiting input similarity. In *Proceedings of the 45th Annual International Symposium on Computer Architecture*. IEEE Press, 57–68.
- [29] Mike Schuster and Kuldip K Paliwal. 1997. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing* 45, 11 (1997), 2673–2681.
- [30] Franyell Silfa, Gem Dot, Jose-Maria Arnau, and Antonio Gonz3lez. 2018. E-PUR: An Energy-efficient Processing Unit for Recurrent Neural Networks. In *Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques (PACT '18)*. ACM, New York, NY, USA, Article 18, 12 pages. <https://doi.org/10.1145/3243176.3243184>
- [31] Avinash Sodani and Gurindar S. Sohi. 1997. Dynamic Instruction Reuse (*ISCA '97*). 194–205. <https://doi.org/10.1145/264107.264200>
- [32] Oriol Vinyals, Alexander Toshev, Samy Bengio, and Dumitru Erhan. 2015. Show and Tell: A Neural Image Caption Generator. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [33] Oriol Vinyals, Alexander Toshev, Samy Bengio, and Dumitru Erhan. 2016. Show and Tell: Lessons learned from the 2015 MSCOCO Image Captioning Challenge. *CoRR abs/1609.06647* (2016). <http://arxiv.org/abs/1609.06647>
- [34] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. 2016. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144* (2016).
- [35] Haiying Xu, Christopher J. F. Pickett, and Clark Verbrugge. 2007. Dynamic Purity Analysis for Java Programs (*PASTE '07*). 75–82. <https://doi.org/10.1145/1251535.1251548>
- [36] Qian Zhang, Ting Wang, Ye Tian, Feng Yuan, and Qiang Xu. 2015. Approx-ANN: An Approximate Computing Framework for Artificial Neural Network. In *Proceedings of the 2015 Design, Automation & Test in Europe Conference & Exhibition (DATE '15)*. EDA Consortium, San Jose, CA, USA, 701–706. <http://dl.acm.org/citation.cfm?id=2755753.2755913>